

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN
BỘ MÔN: CÁC HỆ THỐNG PHÂN TÁN
ĐỀ TÀI: PHÁT TRIỂN HỆ THỐNG CHIA SẺ FILE P2P
(MÔ PHỎNG BITTORRENT)

Giảng viên hướng dẫn: TS. Kim Ngọc Bách

Lớp : HTTT04

Thành viên : Nguyễn Sỹ Huy Kiên - B21DCCN464

Nguyễn Quang Tự - B22DCCN774

Hà Nội - 2026

LỜI CẢM ƠN

Trước tiên, chúng em xin bày tỏ lòng biết ơn sâu sắc đến thầy Kim Ngọc Bách – giảng viên bộ môn Các hệ thống phân tán, người đã tận tình hướng dẫn, hỗ trợ và đưa ra những góp ý quý báu trong suốt quá trình thực hiện đề tài. Những kiến thức chuyên môn cùng sự hướng dẫn của thầy đã giúp chúng em hiểu rõ hơn về hệ thống phân tán, mạng Peer-to-Peer cũng như cách xây dựng một hệ thống mô phỏng BitTorrent hoàn chỉnh. Đồng thời, những nhận xét và góp ý của thầy đã giúp nhóm hoàn thiện bài báo cáo một cách tốt hơn cả về nội dung lẫn hình thức trình bày.

Chúng em cũng xin gửi lời cảm ơn chân thành đến các thành viên trong nhóm đã luôn đồng hành, hỗ trợ và phối hợp chặt chẽ trong suốt quá trình thực hiện đề tài. Mỗi thành viên đều có những đóng góp quan trọng từ việc tìm hiểu lý thuyết, xây dựng hệ thống, kiểm thử chương trình cho đến hoàn thiện báo cáo. Tinh thần hợp tác, trách nhiệm và sự cố gắng của từng thành viên chính là yếu tố giúp nhóm hoàn thành đề tài đúng tiến độ.

Trong quá trình thực hiện, mặc dù nhóm đã cố gắng nghiên cứu và hoàn thiện hệ thống một cách tốt nhất, tuy nhiên do giới hạn về thời gian, kiến thức và kinh nghiệm thực tế nên bài báo cáo khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp và nhận xét từ thầy để có thể tiếp tục hoàn thiện hơn trong tương lai.

Xin chân thành cảm ơn!

MỤC LỤC

LỜI CẢM ƠN.....	2
MỤC LỤC	3
DANH MỤC THUẬT NGỮ VIẾT TẮT	4
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI	5
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	9
CHƯƠNG 3. PHÂN TÍCH THIẾT KẾ HỆ THỐNG	16
CHƯƠNG 4. CÀI ĐẶT VÀ HƯỚNG DẪN SỬ DỤNG.....	28
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	31
TÀI LIỆU THAM KHẢO	36
MÃ NGUỒN DỰ ÁN	37

DANH MỤC THUẬT NGỮ VIẾT TẮT

Từ viết tắt	Ý nghĩa
P2P	Peer-to-Peer
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
IP	Internet Protocol
SHA-256	Secure Hash Algorithm 256-bit
GUI	Graphical User Interface
API	Application Programming Interface
JSON	JavaScript Object Notation
DHT	Distributed Hash Table
RAM	Random Access Memory
LAN	Local Area Network
UI	User Interface
CLI	Command Line Interface
HTTP	HyperText Transfer Protocol
TLS	Transport Layer Security
CDN	Content Delivery Network

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1. Giới thiệu chung

Trong thời đại công nghệ số hiện nay, nhu cầu trao đổi và chia sẻ dữ liệu qua mạng Internet ngày càng tăng mạnh. Các hệ thống truyền tải dữ liệu truyền thống theo mô hình Client–Server đã được sử dụng phổ biến trong nhiều năm nhờ tính đơn giản và dễ quản lý. Tuy nhiên, khi số lượng người dùng tăng cao và dung lượng dữ liệu ngày càng lớn, mô hình này bắt đầu bộc lộ nhiều hạn chế như quá tải máy chủ, chi phí băng thông lớn, tốc độ truyền tải giảm và khả năng mở rộng không cao.

Để giải quyết những vấn đề trên, mô hình mạng ngang hàng Peer-to-Peer (P2P) đã ra đời và trở thành một giải pháp hiệu quả trong việc chia sẻ dữ liệu phân tán. Khác với mô hình Client–Server truyền thống, trong mạng P2P mỗi nút mạng (peer) vừa đóng vai trò là client vừa đóng vai trò là server. Các peer có thể trực tiếp trao đổi dữ liệu với nhau mà không phụ thuộc hoàn toàn vào một máy chủ trung tâm. Điều này giúp giảm tải cho server, tăng khả năng mở rộng hệ thống và tận dụng tối đa tài nguyên mạng của các thiết bị tham gia.

Một trong những giao thức P2P nổi tiếng và thành công nhất là BitTorrent. BitTorrent hoạt động dựa trên cơ chế chia file thành nhiều phần nhỏ gọi là chunk hoặc piece. Người dùng có thể tải đồng thời nhiều chunk từ nhiều peer khác nhau thay vì tải toàn bộ file từ một nguồn duy nhất. Đặc biệt, trong quá trình tải xuống, peer đồng thời cũng chia sẻ lại những chunk đã có cho các peer khác. Chính cơ chế này giúp hệ thống càng có nhiều người tham gia thì tốc độ truyền tải càng được cải thiện.

Xuất phát từ những đặc điểm đó, nhóm lựa chọn đề tài “Xây dựng hệ thống chia sẻ file ngang hàng (P2P) mô phỏng BitTorrent bằng Python” nhằm nghiên cứu và triển khai thực tế cơ chế hoạt động của mạng P2P. Đề tài không chỉ giúp hiểu rõ nguyên lý hoạt động của BitTorrent mà còn là cơ hội để áp dụng các kiến thức về lập trình mạng, hệ

thống phân tán, đa luồng, xử lý đồng thời và kiểm tra toàn vẹn dữ liệu trong môi trường thực tế.

1.2. Lý do chọn đề tài

Hiện nay, các hệ thống phân tán và công nghệ truyền tải dữ liệu ngang hàng đang được ứng dụng rộng rãi trong nhiều lĩnh vực như chia sẻ file, streaming video, blockchain, điện toán đám mây và hệ thống lưu trữ phân tán. Trong đó, mô hình Peer-to-Peer được đánh giá là một trong những kiến trúc mạng có tính mở rộng cao và tối ưu tài nguyên hiệu quả.

Trong quá trình học tập môn Các hệ thống phân tán, nhóm nhận thấy rằng phần lớn kiến thức lý thuyết về socket, giao thức mạng, đồng bộ dữ liệu hay xử lý đa luồng thường khá khó hình dung nếu chỉ học trên tài liệu. Vì vậy, việc xây dựng một hệ thống mô phỏng BitTorrent sẽ giúp chuyển hóa kiến thức lý thuyết thành một sản phẩm thực tế, từ đó hiểu sâu hơn về cách các hệ thống phân tán vận hành trong thực tế.

Ngoài ra, đề tài này còn có nhiều điểm phù hợp để nghiên cứu và phát triển:

- Cho phép áp dụng kiến thức lập trình socket TCP trong truyền thông mạng.
- Áp dụng cơ chế đa luồng để xử lý nhiều kết nối đồng thời.
- Mô phỏng cơ chế phân tán dữ liệu trên nhiều peer.
- Thực hiện kiểm tra tính toàn vẹn dữ liệu bằng thuật toán băm SHA-256.
- Thiết kế giao thức truyền thông riêng giữa các peer và tracker.
- Hiểu rõ cơ chế hoạt động của BitTorrent — một giao thức phổ biến trên Internet.

Bên cạnh yếu tố học thuật, đề tài còn có tính thực tiễn cao vì các nguyên lý của hệ thống P2P hiện nay vẫn đang được sử dụng trong nhiều công nghệ hiện đại như hệ thống CDN, blockchain hay lưu trữ phi tập trung.

1.3. Mục tiêu của đề tài

Mục tiêu chính của đề tài là xây dựng một hệ thống chia sẻ file ngang hàng theo mô hình Peer-to-Peer có khả năng mô phỏng các chức năng cơ bản của giao thức BitTorrent. Thông qua việc triển khai hệ thống, đề tài hướng tới việc áp dụng các kiến thức về lập trình mạng, hệ thống phân tán và xử lý đồng thời vào một bài toán thực tế.

Hệ thống được xây dựng cần cho phép chia file thành nhiều chunk nhỏ để phục vụ quá trình truyền tải phân tán giữa các peer. Đồng thời, hệ thống phải hỗ trợ tạo metadata chứa thông tin về file, kích thước, danh sách chunk và giá trị hash nhằm phục vụ việc kiểm tra tính toàn vẹn dữ liệu trong quá trình truyền tải.

Bên cạnh đó, đề tài cũng tập trung xây dựng Tracker Server có nhiệm vụ quản lý danh sách các peer đang tham gia mạng và hỗ trợ peer discovery. Các peer trong hệ thống cần có khả năng vừa tải file vừa chia sẻ lại các chunk đã có cho các peer khác nhằm mô phỏng đúng cơ chế hoạt động của BitTorrent thực tế.

Ngoài các chức năng cơ bản, hệ thống còn được mở rộng với một số tính năng nâng cao như tải song song từ nhiều peer, thống kê tốc độ upload/download, mô phỏng peer join/leave và thuật toán rarest-first để tối ưu khả năng phân phối dữ liệu trong mạng P2P.

1.4. Phạm vi nghiên cứu

Đề tài tập trung nghiên cứu và triển khai các chức năng cốt lõi của hệ thống chia sẻ file ngang hàng theo mô hình BitTorrent trong môi trường mạng cục bộ. Hệ thống được xây dựng bằng ngôn ngữ Python sử dụng giao thức TCP Socket để thực hiện giao tiếp giữa các peer và tracker.

Trong phạm vi nghiên cứu, đề tài thực hiện các chức năng chính như xây dựng Tracker Server quản lý danh sách peer, cơ chế chia file thành nhiều chunk, tải song song

từ nhiều peer, kiểm tra tính toàn vẹn dữ liệu bằng SHA-256 và xử lý đa luồng trong quá trình upload/download.

Ngoài ra, hệ thống còn hỗ trợ một số chức năng mở rộng như web dashboard theo dõi trạng thái peer theo thời gian thực, mô phỏng peer churn và thuật toán rarest-first nhằm tăng hiệu quả phân phối chunk trong mạng.

Tuy nhiên, do giới hạn về thời gian và phạm vi thực hiện, đề tài chưa triển khai các cơ chế phức tạp của BitTorrent thực tế như Distributed Hash Table (DHT), NAT Traversal, mã hóa TLS hay tối ưu truyền tải trên quy mô Internet lớn. Hệ thống chủ yếu phục vụ mục đích học tập, nghiên cứu và mô phỏng nguyên lý hoạt động của mạng P2P.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Hệ thống phân tán

Trong những năm gần đây, cùng với sự phát triển mạnh mẽ của Internet và điện toán đám mây, các hệ thống phân tán đã trở thành nền tảng quan trọng trong nhiều lĩnh vực công nghệ thông tin. Những hệ thống như mạng xã hội, dịch vụ lưu trữ trực tuyến, streaming video hay blockchain đều được xây dựng dựa trên nguyên lý phân tán nhằm đáp ứng số lượng lớn người dùng và dữ liệu.

Hệ thống phân tán (Distributed System) là tập hợp nhiều máy tính độc lập kết nối với nhau thông qua mạng và phối hợp hoạt động như một hệ thống thống nhất. Mỗi nút trong hệ thống có thể xử lý dữ liệu, lưu trữ tài nguyên và giao tiếp với các nút khác nhằm hoàn thành một nhiệm vụ chung.

Khác với hệ thống tập trung truyền thống, hệ thống phân tán không phụ thuộc hoàn toàn vào một máy chủ duy nhất. Dữ liệu và quá trình xử lý được phân bố trên nhiều nút khác nhau. Điều này giúp hệ thống có khả năng mở rộng cao hơn, tăng hiệu năng xử lý và giảm nguy cơ hệ thống ngừng hoạt động khi một nút gặp sự cố.

Một số đặc điểm quan trọng của hệ thống phân tán bao gồm:

- Nhiều nút hoạt động đồng thời.
- Dữ liệu được phân tán trên nhiều máy.
- Các nút giao tiếp với nhau thông qua mạng.
- Hệ thống có khả năng mở rộng linh hoạt.
- Có khả năng chịu lỗi tốt hơn hệ thống tập trung.

Trong đề tài này, hệ thống chia sẻ file P2P chính là một ví dụ điển hình của hệ thống phân tán. Dữ liệu không được lưu trữ trên một máy chủ duy nhất mà được phân tán trên nhiều peer khác nhau. Mỗi peer có thể tham gia vào quá trình truyền tải dữ liệu, góp phần tăng hiệu năng và khả năng mở rộng của toàn hệ thống.

2.2. Mô hình mạng Peer-to-Peer (P2P)

Peer-to-Peer (P2P) là mô hình mạng trong đó các thiết bị tham gia có vai trò ngang hàng với nhau. Mỗi peer trong hệ thống đều có thể vừa đóng vai trò là client để yêu cầu dữ liệu, vừa đóng vai trò là server để cung cấp dữ liệu cho các peer khác. Điều này khác biệt hoàn toàn so với mô hình Client–Server truyền thống, nơi mọi yêu cầu đều phải đi qua máy chủ trung tâm.

Trong mô hình Client–Server, toàn bộ dữ liệu thường được lưu trữ trên server. Khi số lượng người dùng tăng cao, server rất dễ trở thành điểm nghẽn của hệ thống vì phải xử lý quá nhiều yêu cầu cùng lúc. Nếu server gặp sự cố hoặc mất kết nối, toàn bộ hệ thống cũng có thể ngừng hoạt động.

Ngược lại, mạng P2P cho phép dữ liệu được chia sẻ trực tiếp giữa các peer. Mỗi peer khi tham gia mạng đều đóng góp một phần tài nguyên như băng thông, dung lượng lưu trữ hoặc dữ liệu đã tải về. Chính cơ chế này giúp hệ thống có khả năng mở rộng rất tốt. Số lượng peer càng nhiều thì khả năng chia sẻ dữ liệu càng cao và tốc độ truyền tải cũng có thể được cải thiện.

Ngoài ưu điểm về khả năng mở rộng và tối ưu tài nguyên mạng, mô hình P2P còn giúp giảm tải đáng kể cho server trung tâm. Trong nhiều hệ thống P2P hiện đại, server chỉ đóng vai trò hỗ trợ quản lý hoặc định tuyến, còn dữ liệu thực tế vẫn được truyền trực tiếp giữa các peer.

Tuy nhiên, mô hình này cũng tồn tại nhiều thách thức. Các peer có thể tham gia hoặc rời mạng bất kỳ lúc nào, gây khó khăn trong việc quản lý dữ liệu và duy trì kết nối ổn định. Ngoài ra, hệ thống cũng cần cơ chế kiểm tra tính toàn vẹn dữ liệu nhằm tránh trường hợp peer gửi dữ liệu lỗi hoặc giả mạo.

Trong đề tài này, hệ thống được triển khai theo mô hình P2P có sử dụng Tracker Server. Tracker chỉ chịu trách nhiệm lưu danh sách các peer đang hoạt động và hỗ trợ peer discovery, trong khi quá trình truyền tải file diễn ra trực tiếp giữa các peer.

2.3. Giao thức BitTorrent

BitTorrent là một giao thức chia sẻ file theo mô hình Peer-to-Peer được phát triển vào năm 2001 bởi Bram Cohen. Đây là một trong những giao thức P2P phổ biến nhất trên Internet nhờ khả năng phân phối dữ liệu hiệu quả và tiết kiệm băng thông.

Khác với cách tải file truyền thống từ một server duy nhất, BitTorrent chia file thành nhiều phần nhỏ gọi là chunk hoặc piece. Khi người dùng tải file, hệ thống sẽ tải các chunk này từ nhiều peer khác nhau cùng lúc. Trong quá trình tải xuống, peer cũng đồng thời chia sẻ lại các chunk đã có cho các peer khác. Nhờ đó, số lượng người tham gia càng lớn thì khả năng phân phối dữ liệu càng hiệu quả.

Trong hệ thống BitTorrent tồn tại một số thành phần quan trọng. Seeder là peer đã sở hữu đầy đủ file và có nhiệm vụ chia sẻ dữ liệu cho các peer khác. Leecher là peer đang trong quá trình tải file và chưa có đầy đủ dữ liệu. Tracker là máy chủ quản lý danh sách các peer tham gia torrent và hỗ trợ peer discovery. Ngoài ra, torrent file đóng vai trò là metadata chứa thông tin mô tả file như tên file, kích thước, số lượng chunk và hash của từng chunk.

Quy trình hoạt động của BitTorrent bắt đầu khi seeder chia file thành nhiều chunk nhỏ và tạo file torrent chứa metadata của file. Sau đó, seeder đăng ký với tracker để thông báo rằng mình đang chia sẻ file này. Khi một leecher muốn tải file, peer sẽ đọc torrent file và gửi yêu cầu đến tracker để lấy danh sách các peer đang sở hữu dữ liệu. Sau khi nhận được danh sách peer, leecher sẽ kết nối trực tiếp đến nhiều peer khác nhau để tải các chunk song song.

Sau mỗi chunk tải về, hệ thống sẽ tính toán giá trị hash và so sánh với hash gốc được lưu trong torrent file nhằm kiểm tra tính toàn vẹn dữ liệu. Khi tải đủ tất cả chunk, hệ thống sẽ ghép chúng lại thành file hoàn chỉnh giống với file ban đầu.

Trong đề tài này, hệ thống được xây dựng nhằm mô phỏng các cơ chế cốt lõi của BitTorrent như chia file thành chunk, tải song song từ nhiều peer, kiểm tra hash và cho phép peer vừa tải vừa chia sẻ dữ liệu.

2.4. TCP Socket

Socket là cơ chế cho phép các tiến trình giao tiếp với nhau thông qua mạng máy tính. Trong lập trình mạng, socket được xem như điểm cuối của quá trình truyền dữ liệu giữa hai thiết bị.

Đề tài sử dụng TCP Socket để thực hiện giao tiếp giữa peer với tracker và giữa peer với peer. TCP (Transmission Control Protocol) là giao thức truyền tải hướng kết nối, có khả năng đảm bảo dữ liệu được truyền đầy đủ, đúng thứ tự và hạn chế mất mát dữ liệu trong quá trình truyền tải.

Trước khi gửi dữ liệu, TCP sẽ thực hiện quá trình thiết lập kết nối giữa hai bên thông qua cơ chế bắt tay ba bước (three-way handshake). Sau khi kết nối được thiết lập, dữ liệu sẽ được truyền đi theo thứ tự chính xác và TCP sẽ tự động gửi lại dữ liệu nếu phát hiện mất gói tin.

Việc sử dụng TCP phù hợp với hệ thống truyền file vì dữ liệu cần được đảm bảo chính xác tuyệt đối. Nếu một chunk bị lỗi hoặc thiếu dữ liệu, file sau khi ghép lại sẽ không còn chính xác như file gốc ban đầu.

Trong hệ thống, Tracker Server sử dụng socket để lắng nghe các peer kết nối đến đăng ký hoặc yêu cầu danh sách peer. Đồng thời, mỗi peer cũng mở một socket server riêng để phục vụ upload chunk cho các peer khác trong mạng.

Quy trình hoạt động cơ bản của TCP Socket bao gồm tạo socket, bind địa chỉ IP và port, lắng nghe kết nối, chấp nhận kết nối, gửi nhận dữ liệu và đóng kết nối khi hoàn thành.

2.5. Đa luồng và xử lý đồng thời

Trong hệ thống BitTorrent, nhiều hoạt động cần diễn ra cùng lúc như tải chunk từ nhiều peer, upload chunk cho peer khác hoặc gửi heartbeat tới tracker. Nếu toàn bộ hệ thống chỉ hoạt động trên một luồng duy nhất thì hiệu năng sẽ rất thấp và không thể tận dụng được khả năng tải song song của mô hình P2P.

Để giải quyết vấn đề này, đề tài sử dụng kỹ thuật đa luồng (multithreading). Thread là đơn vị thực thi nhỏ bên trong một tiến trình, cho phép nhiều tác vụ được xử lý đồng thời trong cùng một chương trình.

Trong hệ thống, Tracker Server tạo một thread riêng cho mỗi peer kết nối nhằm cho phép nhiều peer giao tiếp đồng thời. Downloader cũng tạo nhiều thread khác nhau để tải nhiều chunk song song từ nhiều peer. Đồng thời, uploader tạo thread riêng để phục vụ nhiều yêu cầu upload cùng lúc.

Việc sử dụng đa luồng giúp cải thiện đáng kể tốc độ tải file và tăng khả năng xử lý đồng thời của hệ thống. Tuy nhiên, khi nhiều thread cùng truy cập vào một tài nguyên chung, hệ thống có thể gặp phải race condition khiến dữ liệu bị sai lệch hoặc trùng lặp.

Để hạn chế vấn đề này, hệ thống sử dụng cơ chế khóa (`threading.Lock`) nhằm đảm bảo tại một thời điểm chỉ có một thread được phép thay đổi trạng thái của chunk. Điều này giúp PieceManager hoạt động an toàn trong môi trường đa luồng.

2.6. Thuật toán băm SHA-256

SHA-256 là một thuật toán băm thuộc họ SHA-2 được sử dụng phổ biến trong các hệ thống bảo mật và kiểm tra tính toàn vẹn dữ liệu. Thuật toán này có khả năng chuyển đổi dữ liệu đầu vào thành một chuỗi hash có độ dài cố định gồm 256 bit.

Một đặc điểm quan trọng của thuật toán băm là chỉ cần dữ liệu thay đổi rất nhỏ thì giá trị hash cũng sẽ thay đổi hoàn toàn. Nhờ đặc tính này, SHA-256 thường được sử dụng để kiểm tra dữ liệu có bị thay đổi hay không.

Trong hệ thống BitTorrent, SHA-256 được sử dụng để tạo hash cho từng chunk của file. Khi seeder tạo torrent file, hệ thống sẽ tính toán hash cho từng chunk và lưu các giá trị này vào metadata. Sau khi leecher tải chunk về, hệ thống tiếp tục tính hash của chunk vừa nhận và so sánh với hash gốc.

Nếu hai giá trị hash giống nhau, chunk được xem là hợp lệ. Ngược lại, chunk sẽ bị loại bỏ và tải lại từ peer khác. Cơ chế này giúp đảm bảo file sau khi ghép lại hoàn toàn giống với file gốc ban đầu và không bị lỗi trong quá trình truyền tải.

2.7. JSON và giao thức truyền thông

Để các peer và tracker có thể giao tiếp với nhau, hệ thống cần một giao thức truyền thông thống nhất. Trong đề tài này, dữ liệu được trao đổi dưới dạng JSON thông qua TCP Socket.

JSON (JavaScript Object Notation) là định dạng dữ liệu dạng text có cấu trúc đơn giản, dễ đọc và dễ xử lý trong nhiều ngôn ngữ lập trình. Nhờ tính linh hoạt và phổ biến, JSON thường được sử dụng trong các hệ thống truyền thông mạng hiện đại.

Mỗi message trong hệ thống bao gồm loại message và dữ liệu đi kèm. Ví dụ, peer có thể gửi message REGISTER để đăng ký với tracker hoặc gửi REQUEST để yêu cầu một chunk cụ thể từ peer khác.

Việc chuẩn hóa giao thức truyền thông giúp các module trong hệ thống có thể giao tiếp thông nhất và dễ dàng mở rộng trong tương lai. Đồng thời, việc sử dụng JSON cũng giúp quá trình debug và kiểm tra dữ liệu truyền tải trở nên trực quan hơn trong quá trình phát triển hệ thống.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc tổng thể

Hệ thống được xây dựng theo mô hình Peer-to-Peer có sử dụng Tracker Server trung tâm để hỗ trợ quản lý danh sách peer và thực hiện peer discovery. Trong kiến trúc này, tracker không trực tiếp lưu trữ dữ liệu file mà chỉ đóng vai trò như một bộ điều phối giúp các peer tìm thấy nhau trong mạng.

Mỗi peer trong hệ thống đều có khả năng vừa tải dữ liệu vừa chia sẻ dữ liệu cho các peer khác. Điều này giúp hệ thống tận dụng được tài nguyên của toàn bộ các peer tham gia thay vì phụ thuộc hoàn toàn vào một máy chủ trung tâm như mô hình Client-Server truyền thống.

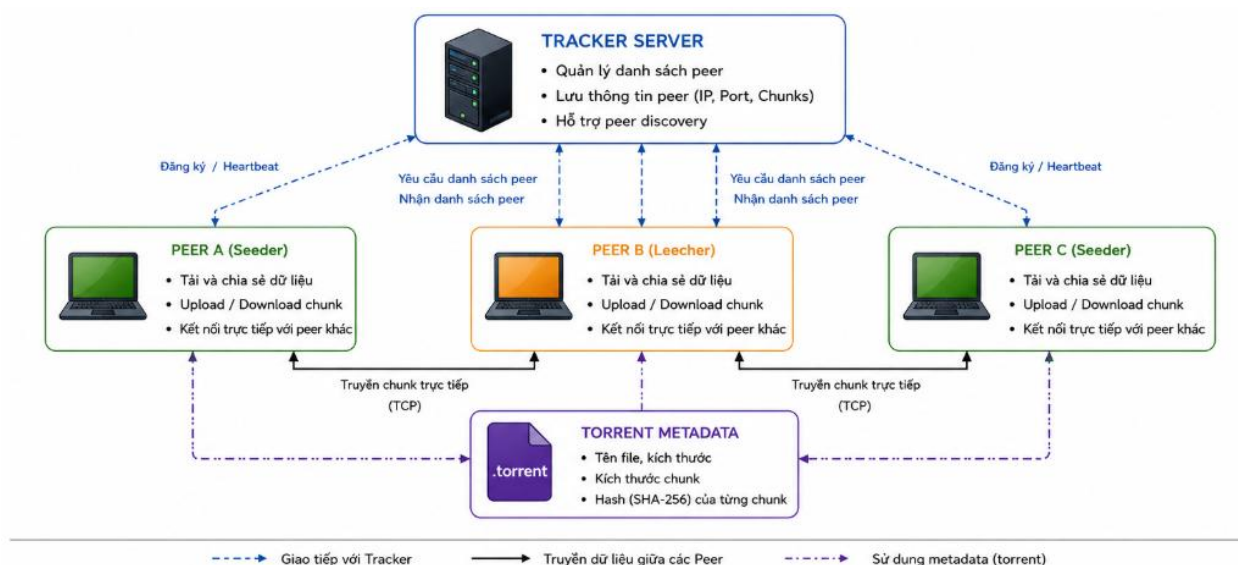
Kiến trúc tổng thể của hệ thống gồm ba thành phần chính:

- Tracker Server,
- Peer Node,
- Torrent Metadata.

Tracker Server chịu trách nhiệm quản lý danh sách các peer đang hoạt động trong mạng. Khi một peer tham gia hệ thống, peer sẽ gửi thông tin đăng ký tới tracker để thông báo rằng mình đang chia sẻ torrent nào và đang sở hữu những chunk nào. Khi một peer khác muốn tải file, peer sẽ gửi yêu cầu tới tracker để lấy danh sách các peer hiện đang có dữ liệu.

Peer Node là thành phần chính tham gia vào quá trình truyền tải dữ liệu. Mỗi peer vừa đóng vai trò là client để tải dữ liệu, vừa đóng vai trò là server để upload chunk cho các peer khác. Các peer giao tiếp trực tiếp với nhau thông qua TCP Socket nhằm truyền tải chunk.

Torrent Metadata là file mô tả thông tin của file cần tải. Metadata chứa tên file, kích thước file, số lượng chunk, kích thước mỗi chunk và giá trị hash của từng chunk nhằm phục vụ việc kiểm tra toàn vẹn dữ liệu.



Kiến trúc tổng thể hệ thống

3.2. Luồng hoạt động của hệ thống

Quy trình hoạt động của hệ thống được xây dựng dựa trên cơ chế hoạt động của BitTorrent. Khi một peer muốn chia sẻ file, hệ thống sẽ chia file thành nhiều chunk nhỏ và tạo torrent metadata chứa thông tin mô tả file.

Đầu tiên, seeder sử dụng chương trình `create_torrent.py` để chia file thành các chunk có kích thước cố định. Sau khi chia xong, hệ thống sẽ tính toán giá trị SHA-256 cho từng chunk và lưu các giá trị hash này vào file torrent.

Tiếp theo, seeder đăng ký với Tracker Server để thông báo rằng mình đang chia sẻ file tương ứng với torrent đó. Tracker sẽ lưu thông tin peer vào bộ nhớ RAM để phục vụ quá trình peer discovery.

Khi một leecher muốn tải file, peer sẽ đọc torrent metadata và gửi yêu cầu GET_PEERS tới tracker. Tracker sau đó trả về danh sách các peer hiện đang sở hữu dữ liệu.

Sau khi nhận được danh sách peer, leecher sẽ mở nhiều kết nối TCP song song tới các peer khác nhau để tải chunk. Mỗi thread download sẽ phụ trách tải một hoặc nhiều chunk cụ thể nhằm tăng tốc độ truyền tải.

Sau khi tải xong một chunk, hệ thống sẽ thực hiện kiểm tra hash SHA-256 nhằm đảm bảo chunk không bị lỗi hoặc thay đổi trong quá trình truyền tải. Nếu chunk hợp lệ, dữ liệu sẽ được lưu xuống đĩa và peer sẽ thông báo với tracker rằng mình đã sở hữu chunk đó.

Khi tải đủ toàn bộ chunk, hệ thống sẽ ghép các chunk lại thành file hoàn chỉnh giống với file gốc ban đầu. Đồng thời, leecher cũng tự động chuyển thành seeder và tiếp tục chia sẻ dữ liệu cho các peer khác.



Luồng hoạt động tổng quát của hệ thống

3.3. Thiết kế cấu trúc thư mục hệ thống

Hệ thống được chia thành nhiều module nhằm giúp mã nguồn dễ quản lý, dễ mở rộng và thuận tiện cho việc bảo trì. Mỗi module đảm nhiệm một nhiệm vụ riêng biệt trong hệ thống.

```

p2p_torrent/
├── common/                                # Thư viện dùng chung
│   ├── hash_utils.py                    SHA256 hash & verify chunk
│   └── file_handler.py                  Chia/ghép file, đọc/lưu
├── chunk
│   ├── torrent_parser.py                Tạo và đọc file .torrent
│   └── protocol.py                      Định nghĩa giao thức mạng
├── tracker/                              # Tracker Server
│   ├── tracker_server.py                TCP server đa luồng
│   ├── tracker_api.py                   Xử lý từng loại request
│   └── peer_manager.py                   Quản lý danh sách peer
├── trong RAM
│   ├── peer/                             # Peer Node
│   │   └── piece_manager.py              Theo dõi trạng thái từng
├── chunk
│   ├── peer_connection.py                Kết nối TCP peer-to-peer
│   ├── uploader.py                      Upload chunk cho peer khác
│   ├── downloader.py                    Download chunk song song
│   ├── tracker_client.py                 Giao tiếp với tracker
│   └── peer_node.py                      Điều phối toàn bộ peer
├── web/
│   └── dashboard.py                      Web dashboard real-time
├── (Flask)
├── tests/                                # 58 test cases
│   ├── test_module1.py                  9 tests
│   ├── test_module2.py                  18 tests
│   ├── test_module3.py                  18 tests
│   ├── test_module4.py                  8 tests
│   └── test_module5.py                  5 tests
├── main_tracker.py                       # Entry point: chạy tracker
├── main_peer.py                           # Entry point: chạy peer
├── create_torrent.py                      # Tool tạo file .torrent
└── simulate_peers.py                     # Mô phỏng nhiều peer đồng thời

```

Thư mục `common` chứa các module dùng chung cho toàn hệ thống như xử lý hash, thao tác file và giao thức truyền thông. Đây là lớp nền tảng phục vụ cho cả tracker và peer.

Thư mục `tracker` chứa các module liên quan đến Tracker Server. Các module này chịu trách nhiệm xử lý request từ peer, quản lý danh sách peer và duy trì trạng thái của hệ thống.

Thư mục `peer` chứa toàn bộ logic hoạt động của Peer Node như download chunk, upload chunk, quản lý trạng thái piece và giao tiếp với tracker.

Thư mục `web` chứa Flask Dashboard phục vụ việc theo dõi trạng thái hệ thống theo thời gian thực. Dashboard cho phép hiển thị danh sách peer, tốc độ truyền tải và tiến độ download.

Ngoài ra, thư mục `tests` chứa các test case dùng để kiểm tra hoạt động của từng module và toàn bộ hệ thống.

3.4. Thiết kế các module chính

Module 1 — File Handler & Torrent Parser

Nền tảng của toàn bộ hệ thống. Gồm 3 file:

File	Chức năng chính
hash_utils.py	hash_chunk(data): tính SHA256. verify_chunk(data, expected): kiểm tra tính toàn vẹn. hash_file(path): hash toàn bộ file.
file_handler.py	split_file(): chia file thành chunks, trả về metadata. read_chunk(): đọc 1 chunk theo index. save_chunk(): lưu chunk xuống đĩa. merge_chunks(): ghép tất cả chunk thành file.
torrent_parser.py	create_torrent(): tạo file .torrent (JSON) chứa metadata. load_torrent(): đọc .torrent trả về dict. info_hash: SHA256 của metadata — định danh duy nhất cho torrent.

Cấu trúc file .torrent:

```
{
```

```

"name": "video.mp4",
"file_size": 4500476,
"chunk_size": 524288,
"num_chunks": 9,
"info_hash": "a3f1cc9b...",
"tracker_url": "127.0.0.1:6969",
"chunks": [
  {"index": 0, "offset": 0, "size": 524288, "hash": "b94d27..."},
  {"index": 1, "offset": 524288, "size": 524288, "hash": "e3b0c4..."},
  ...
]
}

```

Module 2 — Tracker Server

TCP server đa luồng quản lý danh sách peer. Gồm 3 file:

File	Chức năng chính
peer_manager.py	Lưu dữ liệu peer trong dict Python (RAM). Thread-safe với <code>threading.Lock()</code> . Tự động xóa peer timeout sau 30 giây không heartbeat.
tracker_api.py	Xử lý 6 loại request: REGISTER, GET_PEERS, HAVE, HEARTBEAT, UNREGISTER, ERROR. Tách biệt business logic khỏi network code.
tracker_server.py	Mỗi peer kết nối → spawn 1 thread riêng. <code>SO_REUSEADDR</code> tránh lỗi 'port đang dùng' khi restart. Timeout 60 giây mỗi <code>recv()</code> .

Module 3 — Peer Connection & Protocol

File	Chức năng chính
piece_manager.py	Bitfield theo dõi chunk nào có/chưa có/đang pending. Thread-safe:

	50 thread gọi next_needed() không duplicate. Hỗ trợ rarest-first algorithm (Module 5).
peer_connection.py	Wrap socket thành interface đơn giản. send_msg()/recv_msg(): JSON messages. send_piece()/recv_piece(): binary chunk (header JSON + raw bytes). Handshake 2 chiều trao đổi bitfield. Thống kê real-time: bytes sent/received, KB/s.

Module 4 — Peer Node

File	Chức năng chính
uploader.py	Lắng nghe kết nối từ peer khác. Mỗi peer kết nối → 1 thread phục vụ. Đọc chunk từ file gốc hoặc chunk_dir. Verify hash trước khi gửi.
downloader.py	Tải chunk từ nhiều peer song song. Mỗi peer → 1 thread riêng. Hash verify sau mỗi chunk. Callback on_chunk_complete báo tracker.
tracker_client.py	Giao tiếp với tracker qua TCP ngắn hạn. Heartbeat tự động mỗi 10 giây trong thread nền.
peer_node.py	Điều phối toàn bộ: seed mode và download mode. Sau khi tải xong tự chuyển thành seeder.

Module 5 — Tích hợp & Nâng cao

Tính năng	Mô tả
Rarest-first algorithm	Ưu tiên tải chunk mà ít peer nhất đang có, tăng availability cho toàn mạng
Leecher → Seeder	Sau khi tải xong, peer tự đăng ký lại tracker với đầy đủ chunk, phục vụ peer khác
Heartbeat timeout	Tracker tự xóa peer không gửi heartbeat trong 30 giây, danh sách luôn chính xác
Web Dashboard	Flask server hiển thị real-time: số peer, vai trò, tiến độ từng chunk

Peer Churn simulation	simulate_peers.py mô phỏng peer join/leave giữa chừng, hệ thống vẫn hoạt động
-----------------------	---

3.5. Thiết kế giao thức truyền thông

Hệ thống sử dụng giao thức truyền thông tự thiết kế dựa trên JSON thông qua TCP Socket. Mỗi message bao gồm loại message (`type`) và dữ liệu đi kèm (`data`).

Ví dụ một message đăng ký peer:

```
{
  "type": "REGISTER",
  "data": {
    "peer_id": "peer01",
    "port": 7001
  }
}
```

Việc sử dụng JSON giúp message dễ đọc, dễ debug và dễ mở rộng thêm các trường dữ liệu mới trong tương lai.

Hệ thống định nghĩa nhiều loại message khác nhau phục vụ cho các chức năng: đăng ký peer, tìm kiếm peer, yêu cầu chunk, gửi chunk, cập nhật trạng thái chunk, heartbeat kiểm tra kết nối.

Message	Chiều	Ý nghĩa	Payload
REGISTER	Peer → Tracker	Đăng ký tham gia mạng	info_hash, peer_id, port, chunks[]
GET_PEERS	Peer → Tracker	Hỏi ai đang có torrent X	info_hash, peer_id

PEER_LIST	Tracker → Peer	Danh sách peer đang có	peers[], count
HAVE	Peer → Tracker	Vừa tải xong chunk N	info_hash, peer_id, chunk_index
HEARTBEAT	Peer → Tracker	Tôi vẫn còn online	info_hash, peer_id
UNREGISTER	Peer → Tracker	Tôi rời mạng	info_hash, peer_id
BITFIELD	Peer → Peer	Trao đổi danh sách chunk khi kết nối	peer_id, info_hash, chunks[]
REQUEST	Peer → Peer	Yêu cầu 1 chunk cụ thể	chunk_index
PIECE	Peer → Peer	Gửi chunk (header JSON + binary data)	index, size + raw bytes
BYE	Peer → Peer	Ngắt kết nối lịch sự	{}
OK	Tracker → Peer	Xác nhận thành công	message
ERROR	Tracker → Peer	Thông báo lỗi	message

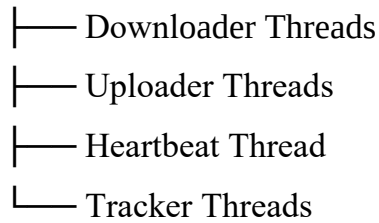
3.6. Thiết kế xử lý đa luồng

Để tăng tốc độ truyền tải dữ liệu, hệ thống sử dụng multithreading trong cả tracker và peer.

Tracker Server tạo một thread riêng cho mỗi peer kết nối nhằm cho phép nhiều peer giao tiếp đồng thời. Trong khi đó, downloader cũng tạo nhiều thread để tải nhiều chunk song song từ nhiều peer khác nhau.

Uploader của peer cũng hoạt động theo cơ chế đa luồng. Mỗi khi có peer khác kết nối yêu cầu dữ liệu, hệ thống sẽ tạo một worker thread riêng để phục vụ request đó.

Main Thread



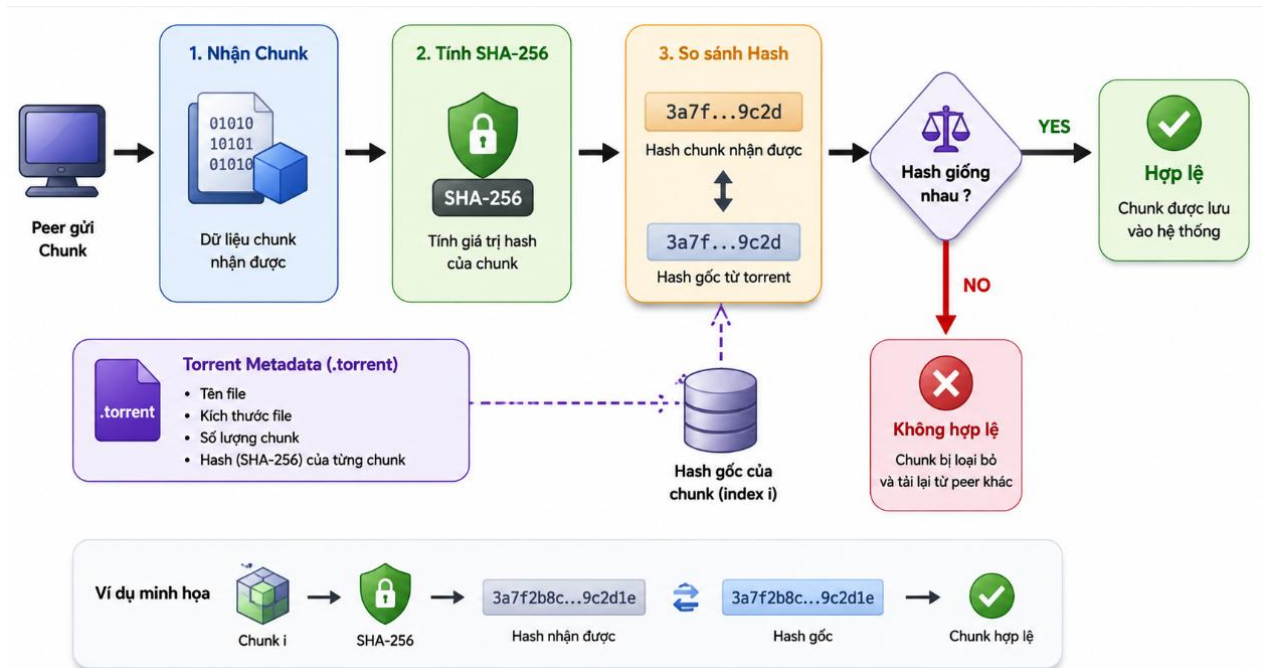
Việc xử lý đa luồng giúp hệ thống tận dụng tốt hơn tài nguyên CPU và tăng hiệu suất truyền tải dữ liệu. Tuy nhiên, để tránh race condition khi nhiều thread cùng truy cập PieceManager, hệ thống sử dụng `threading.Lock()` nhằm đảm bảo tính thread-safe.

3.7. Thiết kế quản lý chunk và hash

Trong hệ thống, file được chia thành nhiều chunk có kích thước cố định nhằm hỗ trợ truyền tải song song và resume download.

Mỗi chunk đều có:

- index,
- offset,
- size,
- hash SHA-256.



Quy trình kiểm tra chunk

Sau khi tải một chunk, hệ thống sẽ tính toán hash SHA-256 của dữ liệu nhận được và so sánh với hash gốc trong torrent metadata. Nếu hash không khớp, chunk sẽ bị loại bỏ và tải lại từ peer khác.

Việc kiểm tra hash giúp đảm bảo dữ liệu sau khi ghép hoàn toàn giống với file gốc ban đầu.

3.8. Thiết kế xử lý lỗi

Tình huống lỗi	Cách xử lý
Peer rút mạng giữa chừng	Thread tải từ peer đó kết thúc, chunk bị mark_failed(), thread khác tải lại từ peer còn lại
Hash chunk sai (bị lỗi/giả mạo)	Bỏ chunk, mark_failed(), tải lại từ peer khác
Không tìm thấy peer nào	Retry tối đa 5 lần, mỗi lần chờ 3 giây

Peer không heartbeat 30 giây	Tracker tự động xóa peer đó khỏi danh sách
Port đã bị dùng	Báo lỗi rõ ràng, không crash silently
File .torrent không đúng format	Validate các trường bắt buộc, raise ValueError rõ ràng

CHƯƠNG 4. CÀI ĐẶT VÀ HƯỚNG DẪN SỬ DỤNG

4.1. Yêu cầu hệ thống

Hệ thống được xây dựng bằng ngôn ngữ Python và không yêu cầu cấu hình phần cứng quá cao. Chương trình có thể hoạt động trên các hệ điều hành phổ biến như Windows hoặc Linux trong môi trường mạng cục bộ.

Để triển khai và chạy hệ thống, máy tính cần cài đặt Python phiên bản 3.10 trở lên. Ngoài ra, hệ thống sử dụng Flask để xây dựng web dashboard phục vụ việc theo dõi trạng thái hoạt động của các peer trong mạng P2P.

Thành phần	Yêu cầu
Ngôn ngữ lập trình	Python 3.10+
Web Framework	Flask
Giao thức mạng	TCP Socket
Database	Không sử dụng
Hệ điều hành	Windows / Linux

Cài đặt Flask bằng lệnh: `pip install flask`

Ngoài Flask, hệ thống không sử dụng thêm thư viện ngoài nào khác. Các chức năng như socket, multithreading, hash SHA-256 và xử lý JSON đều sử dụng thư viện chuẩn của Python.

4.2. Hướng dẫn chạy demo

Bước 1 — Tạo file .torrent:

```
python create_torrent.py video.mp4
```

Bước 2 — Chạy Dashboard (Terminal 1):

```
python web/dashboard.py  
# Mở trình duyệt: http://localhost:5000
```

Bước 3 — Chạy Seeder (Terminal 2):

```
python main_peer.py seed --id seeder_A --port 7001 --file  
video.mp4 --torrent video.mp4.torrent --tracker-port 6969
```

Bước 4 — Chạy Leecher (Terminal 3):

```
python main_peer.py download --id leecher_B --port 7002 --  
torrent video.mp4.torrent --tracker-port 6969
```

Bước 5 — Verify kết quả:

```
python -c "  
from common.hash_utils import hash_file  
print(hash_file('video.mp4') ==  
hash_file('downloads/leecher_B/video.mp4'))  
"  
# Output: True — file giống hệt nhau 100%
```

Bước 6 — Chạy mô phỏng tự động:

```
python simulate_peers.py
```

5.3. Chạy toàn bộ test

Để đảm bảo hệ thống hoạt động ổn định, project được xây dựng kèm theo các module kiểm thử tự động cho từng chức năng.

Các test bao gồm:

- kiểm tra tracker,
- kiểm tra upload/download,
- kiểm tra hash,
- kiểm tra piece manager,
- kiểm tra xử lý lỗi.

```
python -m tests.test_module1    # 9 tests
python -m tests.test_module2    # 18 tests
python -m tests.test_module3    # 18 tests
python -m tests.test_module4    # 8 tests
python -m tests.test_module5    # 5 tests
```

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết quả đã đạt được

Yêu cầu đề bài	Trạng thái	Chi tiết
Chia file thành chunks + metadata	Hoàn thành	split_file() + create_torrent()
Tracker/bootstrap server	Hoàn thành	TrackerServer TCP đa luồng, port 6969
Peer discovery	Hoàn thành	GET_PEERS → PEER_LIST
Tải từ nhiều peer song song	Hoàn thành	Downloader multithreading
Upload chunk cho peer khác	Hoàn thành	Uploader lắng nghe và phục vụ
Kiểm tra toàn vẹn bằng hash	Hoàn thành	SHA256 verify từng chunk
Ghép chunk thành file	Hoàn thành	merge_chunks()
TCP socket / giao thức mạng	Hoàn thành	Giao thức JSON tự thiết kế
Đa luồng / xử lý đồng thời	Hoàn thành	Threading cho cả upload và download
Vừa tải vừa upload	Hoàn thành	Uploader và Downloader chạy song song
Xử lý peer rời mạng	Hoàn thành	mark_failed() + retry + timeout
Rarest-first (nâng cao)	Hoàn thành	next_needed_rarest_first()
Thống kê tốc độ (nâng cao)	Hoàn	avg_speed_kbps, bytes_sent/received

	thành	
Mô phỏng peer churn (nâng cao)	Hoàn thành	simulate_peers.py
Giao diện web/GUI (nâng cao)	Hoàn thành	Flask dashboard real-time

5.2. Ưu điểm

Hệ thống được xây dựng với kiến trúc module hóa rõ ràng giúp mã nguồn dễ quản lý và dễ mở rộng trong tương lai. Các chức năng như tracker, downloader, uploader hay piece manager được tách riêng thành các module độc lập, giúp thuận tiện cho việc bảo trì và phát triển thêm tính năng mới.

Một ưu điểm quan trọng khác là khả năng tải song song từ nhiều peer. Việc chia file thành nhiều chunk nhỏ cho phép hệ thống tận dụng băng thông của nhiều peer cùng lúc, từ đó cải thiện tốc độ truyền tải dữ liệu.

Hệ thống cũng hỗ trợ cơ chế upload/download đồng thời, đúng với đặc điểm của mạng P2P thực tế. Trong khi đang tải file, peer vẫn có thể chia sẻ các chunk đã tải xong cho các peer khác trong mạng.

Bên cạnh đó, việc sử dụng SHA-256 để kiểm tra hash của từng chunk giúp đảm bảo tính toàn vẹn dữ liệu trong quá trình truyền tải. Điều này giúp hạn chế lỗi file hỏng hoặc dữ liệu sai lệch khi truyền qua mạng.

Ngoài ra, hệ thống còn có khả năng: resume download cơ bản, xử lý peer mất kết nối, mô phỏng nhiều peer hoạt động đồng thời, theo dõi trạng thái hệ thống qua dashboard.

5.3. Hạn chế của hệ thống

Mặc dù đã hoàn thành các chức năng chính của hệ thống BitTorrent mô phỏng, đề tài vẫn còn tồn tại một số hạn chế nhất định.

Thứ nhất, hệ thống hiện vẫn sử dụng Tracker Server tập trung. Điều này khiến tracker trở thành điểm lỗi duy nhất (Single Point of Failure). Nếu tracker ngừng hoạt động, các peer mới sẽ không thể thực hiện peer discovery mặc dù dữ liệu vẫn còn tồn tại trên các peer khác.

Thứ hai, hệ thống mới chỉ hoạt động tốt trong môi trường mạng cục bộ hoặc quy mô nhỏ. Khi số lượng peer tăng quá lớn, hiệu năng của tracker và số lượng kết nối socket đồng thời có thể trở thành bottleneck của hệ thống.

Ngoài ra, hệ thống chưa hỗ trợ các cơ chế phức tạp của BitTorrent thực tế như: Distributed Hash Table (DHT), NAT Traversal, mã hóa kết nối TLS, congestion control, cơ chế tit-for-tat tối ưu băng thông.

Việc xử lý đa luồng hiện tại cũng mới dừng ở mức cơ bản. Trong môi trường có số lượng lớn thread hoạt động đồng thời, hệ thống vẫn có nguy cơ xảy ra race condition nếu không được đồng bộ chặt chẽ.

Bên cạnh đó, hệ thống chưa có cơ chế xác thực peer hoặc mã hóa dữ liệu truyền tải nên vẫn tồn tại các rủi ro liên quan đến bảo mật và giả mạo dữ liệu.

5.4. Hướng phát triển

Trong tương lai, hệ thống có thể tiếp tục được mở rộng và nâng cấp để tiệm cận hơn với mô hình BitTorrent thực tế.

Một trong những hướng phát triển quan trọng là thay thế Tracker Server tập trung bằng cơ chế Distributed Hash Table (DHT). Điều này giúp loại bỏ điểm lỗi tập trung và tăng khả năng mở rộng của hệ thống.

Ngoài ra, hệ thống có thể được nâng cấp bằng cách áp dụng các cơ chế:

- NAT Traversal để hỗ trợ kết nối qua Internet,
- mã hóa TLS nhằm tăng tính bảo mật,
- cơ chế xác thực peer,
- thuật toán tit-for-tat để tối ưu chia sẻ băng thông.

Bên cạnh đó, hệ thống cũng có thể chuyển sang sử dụng asynchronous I/O thay cho multithreading truyền thống nhằm cải thiện hiệu năng khi xử lý số lượng lớn kết nối đồng thời.

Dashboard hiện tại cũng có thể được mở rộng thêm các chức năng như:

- biểu đồ tốc độ truyền tải theo thời gian,
- thống kê số lượng chunk,
- theo dõi swarm realtime,
- giám sát peer churn.

Ngoài phạm vi mô phỏng trong mạng cục bộ, hệ thống còn có thể được triển khai thử nghiệm trên môi trường Internet thực tế nhằm đánh giá khả năng hoạt động ở quy mô lớn hơn.

5.5. Kết luận chung

Đề tài “Xây dựng hệ thống chia sẻ file P2P mô phỏng BitTorrent bằng Python” đã hoàn thành các mục tiêu đề ra và mô phỏng thành công cơ chế hoạt động cơ bản của giao thức BitTorrent.

Thông qua quá trình thực hiện đề tài, nhóm không chỉ hiểu rõ hơn về hệ thống phân tán và mạng P2P mà còn tích lũy thêm nhiều kinh nghiệm thực tế trong việc thiết kế và triển khai hệ thống mạng có xử lý đồng thời.

Mặc dù hệ thống vẫn còn nhiều hạn chế so với BitTorrent thực tế, đây vẫn là nền tảng quan trọng để tiếp tục nghiên cứu và phát triển các hệ thống phân tán phức tạp hơn trong tương lai.

DANH MỤC TÀI LIỆU THAM KHẢO

- [1] Distributed Systems: Principles and Paradigms, Andrew S. Tanenbaum, Maarten Van Steen, *Distributed Systems: Principles and Paradigms*, 2nd Edition, Pearson Education, 2007.
- [2] Computer Networking: A Top-Down Approach, James F. Kurose, Keith W. Ross, *Computer Networking: A Top-Down Approach*, 7th Edition, Pearson, 2017.
- [3] [Python Official Documentation](#), Python Software Foundation, truy cập tháng 5 năm 2026.
- [4] [Flask Documentation](#), Pallets Project, truy cập tháng 5 năm 2026.
- [5] [BitTorrent Protocol Specification](#), BitTorrent.org, “The BitTorrent Protocol Specification”, truy cập tháng 5 năm 2026.
- [6] TCP/IP Illustrated, Volume 1, W. Richard Stevens, *TCP/IP Illustrated, Volume 1: The Protocols*, Addison-Wesley, 1994.

MÃ NGUỒN DỰ ÁN

https://github.com/devouring-blade/P2P_BitTorrent

